Laboratory 6:

Correlation & Signal Processing Fundamentals
Correlation-Based Ultrasonic Distance Sensor

Time-Delay Estimation and Edge Computing Trade-Offs

Lab Duration: 2 hours
Pre-Lab Time Expected: 30 minutes (physics and calculations)
Key Concept: Understanding how cross-correlation enables time-delay estimation—a fundamental technique used in autonomous systems, aerospace, and edge computing.

Contents

Grading and Authenticity Requirements

Deliverable (1/2): This is a graded laboratory assignment. Include your **name and student ID** in all required screenshots and photographs.

Deliverable (2/2): All distance measurements must be validated against physical ruler measurements. Pre-computed or unrealistic sensor data is not accepted.

Checkpoint (1/8): Before submission, verify that each MATLAB figure includes axis labels with units, a legend, and your name/student ID in the title (or `sgtitle`).

1 Real-World Context: Cross-Correlation for Embedded Systems

You are a Junior Engineer at the Advanced Systems division of an aerospace, medical device, or robotics company. Your role is to design embedded signal processing systems that extract time delays from noisy acoustic or electromagnetic signals using cross-correlation. The challenge: computing power is limited, latency is critical, and every milliwatt of energy matters. In this section, we explore three real-world programs where your lab skills directly impact production systems.

1.1 Program 1: Stratospheric Intelligence Platforms (Aerospace)

Your company is designing the avionics system for a solar-powered autonomous aircraft operating at 70,000 feet. The platform collects imagery and atmospheric data over contested regions where GPS is spoofable and barometric altimeters drift with weather.

The problem: The aircraft must maintain stable cruise altitude through wind gusts and density changes. How do you measure altitude reliably?

Your solution: Acoustic ranging via cross-correlation. The aircraft transmits a brief ultrasonic chirp downward and listens for the echo from the ground. The time-of-flight equation gives distance:

$$d = \frac{v_{\text{sound}} \cdot \Delta t}{2}. \tag{1}$$

But the received echo is buried in noise from wind, ground reflections, and receiver noise. Cross-correlation acts as a matched filter—it extracts the time delay with precision even in 5 dB SNR conditions. Your embedded implementation must run on a 16-bit microcontroller with 2 KB RAM and deliver the altitude estimate within 100 ms (critical for autopilot stability).

1.2 Program 2: Prenatal Ultrasound Diagnosis (Medical Devices)

Your company manufactures portable ultrasound systems for fetal monitoring in low-resource clinics. These devices must be battery-powered, operate in real-time, and deliver clinically accurate measurements with no connectivity.

The problem: Fetal position and growth require precise distance measurements from the transducer to anatomical landmarks (spine, head). Traditional systems rely on on-board GPU processing, but field deployments need embedded-only solutions.

Your solution: Correlation-based time-delay estimation. The ultrasound transducer transmits a pulse, and the received echoes from tissue boundaries are cross-correlated with a template waveform. The peak correlation lag directly gives distance-to-landmark. Your embedded system must achieve < 0.5 cm accuracy on fetal measurements (regulatory requirement) while running on a low-power ARM Cortex-M4 with 256 KB RAM.

1.3 Program 3: Autonomous Underwater Vehicle Sonar (Robotics)

Your company develops AUVs for deep-sea mineral surveys. The vehicles operate autonomously in darkness with no GPS, relying solely on acoustic sensing for navigation and obstacle avoidance.

The problem: The vehicle must detect seafloor distance, identify geological features, and avoid subsea cables and equipment—all in real-time with latencies < 50 ms.

Your solution: Sonar via cross-correlation. The vehicle's forward-looking sonar transmits a chirp and cross-correlates the received echoes with the transmitted waveform. Multiple receiver elements enable beam-forming and multi-target detection. The correlation must run on an embedded Linux computer with strict power budgets (the vehicle operates 8-hour missions on battery).

1.4 The Common Challenge: Edge vs. Cloud Processing Trade-Offs

All three programs face the same critical decision: **where should signal processing happen?**

- **Cloud/Ground Station:** Raw ADC data streams to powerful computers (MATLAB servers, GPUs, cloud infrastructure). Algorithms are sophisticated (FFT-based detection, adaptive filtering, beamforming). **Advantage:** high accuracy and flexibility. **Disadvantage:** high latency (unacceptable for flight control or obstacle avoidance), high bandwidth (expensive satellite uplinks or sonar data links), high power (satellite modems drain batteries).
- **Edge/On-Board:** Embedded computer (Arduino, ARM Cortex, microcontroller) computes time delay on-the-fly. **Advantage:** immediate decisions (critical for autonomy), low power, low bandwidth, robust to communication dropouts. **Disadvantage:** limited compute means simpler algorithms, lower precision, quantization errors, potential accuracy loss.

Your lab directly simulates this trade-off:

1. **Simulate the ground station:** Use MATLAB's optimized `xcorr()` function with full-precision floating-point arithmetic and signal normalization. This achieves the best possible accuracy—representative of a cloud-based system.
2. **Simulate the edge device:** Implement cross-correlation on the Arduino with integer arithmetic, limited window size, and resource constraints. This reveals real-world accuracy loss.
3. **Analyze the trade-off:** Quantify the accuracy penalty of edge processing. For each real application, decide: can I tolerate the accuracy loss, or must I pay the latency/power/bandwidth cost to use the cloud?

2 Learning Objectives

By the end of this lab, you should be able to:

1. **Understand the physics of sound-based ranging:** Derive the time-of-flight equation $d = v\Delta t/2$ and predict expected time delays for known distances.
2. **Grasp the mathematical foundation of cross-correlation:** Recognize that cross-correlation $R_{xy}[k]$ measures the similarity between two signals as a function of time lag k, and understand how the peak of $R_{xy}[k]$ reveals the time delay between signals.
3. **Implement correlation in two environments:** Write MATLAB code using the `xcorr()` function (high-precision) and then implement a sliding dot-product correlation on the Arduino (resource-constrained).
4. **Convert sample-domain delay to time and distance:** Map the sample index of the correlation peak to actual time using the sampling rate, and convert time to physical distance.
5. **Quantify measurement accuracy:** Validate sensor distances against physical ruler measurements, compute error, and explain discrepancies between the Arduino and MATLAB implementations.
6. **Analyze real-world signal processing trade-offs:** Discuss the bandwidth, latency, power, and accuracy implications of edge versus cloud computing, using your lab results as evidence.

Emphasis: This lab bridges discrete-time signal processing theory with real-world sensor systems and embedded constraints.

3 Physics and Mathematics Background

3.1 Sound Propagation and Time-of-Flight

Sound travels through air at a speed that depends on temperature and pressure. Under standard laboratory conditions (room temperature, sea level), the speed of sound is approximately:

$$v_{\text{sound}} \approx 343 \text{ m/s}. \tag{2}$$

When an acoustic pulse travels from a transmitter to a target (the floor, a wall, an object) and back to a receiver, the total distance traveled is $2d$ (down and back up). If the round-trip takes time Δt, then:

$$2d = v_{\text{sound}} \cdot \Delta t \quad \Rightarrow \quad d = \frac{v_{\text{sound}} \cdot \Delta t}{2}. \tag{3}$$

Example: If the round-trip time is $\Delta t = 0.6$ ms, then:

$$d = \frac{343 \text{ m/s} \times 0.6 \times 10^{-3} \text{ s}}{2} = 0.1029 \text{ m} \approx 10.3 \text{ cm}. \tag{4}$$

3.2 Cross-Correlation: The Mathematical Foundation

Problem: You transmit a known signal $x[n]$ (a chirp), and receive a noisy echo $y[n]$. How do you reliably detect when the echo arrives?

Solution: Compute the cross-correlation between the transmitted and received signals:

$$R_{xy}[k] = \sum_{n=0}^{N-1} x[n] \cdot y[n+k], \tag{5}$$

where k is the lag (time shift in samples).

Interpretation:

- $R_{xy}[k]$ measures how well the transmitted signal $x[n]$ aligns with the received signal $y[n]$ when shifted by k samples.
- When k equals the true delay, the alignment is best (the transmit chirp "matches" the echo), so $R_{xy}[k]$ reaches a maximum.
- The sample index k_{peak} of this maximum directly tells us the time delay: $\Delta t = k_{\text{peak}}/f_s$, where f_s is the sampling rate.

Why does this work? The echo is a delayed, attenuated, and noisy replica of the transmitted signal. Cross-correlation is a matched filter—it extracts the signal of interest from noise and precisely locates its arrival time.

3.3 Discrete-Time Implementation

In the lab, the Arduino and MATLAB both work with discrete-time samples:

- The transmitted signal is stored as an array $x[0], x[1], \ldots, x[N_x - 1]$.
- The received signal is captured as an array $y[0], y[1], \ldots, y[N_y - 1]$.

- The cross-correlation is computed for lags $k = 0, 1, 2, \ldots, N_y - N_x$ (or more, depending on the implementation).
- The peak lag k_{peak} is found by searching for the maximum value in $R_{xy}[k]$.

MATLAB's `xcorr()` function is optimized for speed and accuracy. The Arduino implementation will be simpler but slower, with fewer samples and lower precision.

[INSERT DIAGRAM: Conceptual illustration of transmitted chirp, received echo, and cross-correlation peak]

4 Required Equipment and Software

Table 1. Required Hardware and Software

Category	**Required Items**
Controller	Arduino Uno or Mega, USB cable
Acoustic transducers	Ultrasonic transmitter (40 kHz typical), receiver with pre-amp
Signal generation	Arduino PWM or external function generator (for transmitter drive)
Reception	Arduino ADC pin (A0), 16-bit if available for dynamic range
Measurement	Physical ruler (centimeter markings), multimeter
Prototyping	Breadboard, jumper wires, coupling capacitor (e.g., 10 µF)
Software	Arduino IDE, MATLAB R2021a or later
Provided skeletons	`ChirpGenerate.ino`, `EchoCapture.ino`, `Lab06_Serial_Reader.m`

Arduino Pin Roles

Table 2. Pin Assignments

Signal	**Pin**	**Direction**	**Purpose**
Transmitter drive	D9 or D10	OUTPUT (PWM)	Drives ultrasonic transmitter with 40 kHz signal
Receiver signal	A0	INPUT	Reads amplified echo from receiver
Trigger output	D13	OUTPUT	LED toggles at acquisition start
Serial link	USB	TX/RX	115200 baud stream to MATLAB

5 Safety and Wiring Notes

- **Acoustic safety:** Ultrasonic transmitters operating at 40 kHz are inaudible to human ears but can be uncomfortable at close range. Avoid placing your ear directly in front of the transmitter.
- **Power supply:** The transmitter driver may draw up to 100 mA. Ensure Arduino power supply is adequate (typical USB supplies 500 mA).
- **Coupling capacitor:** A 10 µF capacitor may be needed between the function generator and the transmitter to block DC and pass the AC signal.
- **Receiver amplification:** The raw receiver output is weak (millivolts). Use a pre-amplified receiver board, or amplify with an op-amp circuit.
- **ADC input protection:** Keep A0 within the 0–5 V range at all times.

6 Pre-Lab Preparation (Complete Before Lab Session)

6.1 Speed of Sound and Time-of-Flight Calculations

Pre-lab Deliverable (1/2): Derive the relationship between distance, speed of sound, and round-trip time delay. Then calculate the expected time delays for several reference distances.

Theoretical Derivation

Starting from the definition of speed:

$$v_{\text{sound}} = \frac{\text{distance}}{\text{time}}. \tag{6}$$

For a round-trip echo (transmitter to target and back), the total distance is $2d$. If the round-trip time is Δt:

$$v_{\text{sound}} = \frac{2d}{\Delta t}. \tag{7}$$

Solving for distance:

$$\boxed{d = \frac{v_{\text{sound}} \cdot \Delta t}{2}} \tag{8}$$

Student Notes: Handwritten derivation space:

Reference Time-of-Flight Calculations

Using $v_{\text{sound}} = 343$ m/s, calculate the round-trip time delay for each distance:

Student Notes: Calculated Reference Delays:

For $d = 10$ cm: $\Delta t = 2 \times 0.10/343 =$ _____ ms

For $d = 20$ cm: $\Delta t = 2 \times 0.20/343 =$ _____ ms

For $d = 30$ cm: $\Delta t = 2 \times 0.30/343 =$ _____ ms

For $d = 50$ cm: $\Delta t = 2 \times 0.50/343 =$ _____ ms

For $d = 100$ cm: $\Delta t = 2 \times 1.00/343 =$ _____ ms

Sample-Domain Delay Prediction

The Arduino will sample the received signal at a rate f_s (e.g., 8 kHz or higher). Convert the time delay to sample index:

$$k_{\text{delay}} = \Delta t \cdot f_s. \tag{9}$$

Student Notes: Sample-domain prediction:

If $f_s = 8$ kHz and $\Delta t = 0.583$ ms (for 10 cm), then:
$k_{\text{delay}} = 0.583 \times 10^{-3} \times 8000 =$ _____ samples

Checkpoint (2/8): Show the TA your handwritten derivation and reference calculations for Checkpoint 1 sign-off.

7 Hardware Setup: The Ultrasonic Acoustic Rangefinder

7.1 Transducer Basics

An ultrasonic transducer system consists of:

- **Transmitter:** A piezoelectric element that vibrates at 40 kHz, generating sound waves into the air.
- **Receiver:** Another piezoelectric element that picks up the echo. The received signal is very weak (millivolts) and noisy.
- **Pre-amplifier (optional):** An op-amp circuit that amplifies the received signal to usable levels (0–5 V for the ADC).

7.2 Wiring Configuration

Transmitter Drive:

- Arduino pin D9 or D10 (PWM-capable) generates a 40 kHz square wave or sinusoid.
- This signal is AC-coupled (through a 10 µF capacitor) to the transmitter element.
- The transmitter radiates acoustic energy downward (toward the target).

Receiver Path:

- The receiver element picks up the echo (a very weak signal, often millivolts).
- A pre-amplifier (either on a commercial breakout board or built with an op-amp) amplifies the signal.
- The amplified signal is AC-coupled to Arduino pin A0 (ADC).
- The ADC samples at a high rate (e.g., 8–16 kHz) to capture the full echo transient.

[INSERT ULTRASONIC TRANSDUCER WIRING SCHEMATIC HERE: Show Arduino D9 to transmitter via coupling capacitor, and receiver pre-amplifier output to Arduino A0, with ground connections]

Student Notes: Student wiring checklist:

- ☐ Transmitter coupled to D9 via 10 µF capacitor
- ☐ Receiver pre-amp output connected to A0
- ☐ All grounds connected to Arduino GND
- ☐ Power supply sufficient for pre-amp (typically 5V from Arduino)

Checkpoint (3/8): Before powering on, TA visually inspects all connections and verifies coupling capacitor polarity.

8 Lab Procedure (In-Lab Execution, 2 Hours)

Target Timeline: CP-1 (10 min) → Hardware setup (15 min) → Arduino testing (15 min) → MATLAB analysis (40 min) → Distance validation (15 min) → wrap-up (5 min).

8.1 Part 1: Pre-Lab Check and Hardware Verification

1. Show the TA your handwritten pre-lab calculations (time-of-flight values for 10–100 cm).
2. Set up the breadboard with the transmitter and receiver as shown in the wiring schematic.
3. Use a multimeter to verify power supply to the pre-amplifier.
4. Gently touch the transmitter element—you should feel a faint vibration (do not place ear directly in front).

Checkpoint (4/8): TA verifies wiring, component connections, and signs off on Checkpoint 1.

8.2 Part 2: Arduino Chirp Generation and Echo Capture

Objective: Configure the Arduino to generate a transmit chirp and capture the received echo.

Arduino Skeleton Code Overview

Two skeleton files are provided:

- `ChirpGenerate.ino`: Generates a 40 kHz PWM signal on D9. Simply configure the frequency and amplitude.
- `EchoCapture.ino`: Samples A0 at high speed, stores samples in memory, and streams data to MATLAB via serial.

You do not need to write the PWM or ADC acquisition code. Only edit the configuration section.

Configuration Parameters

```
// USER CONFIGURATION
#define TRANSMIT_PIN    9           // D9 PWM output
#define ADC_PIN         A0          // A0 input
#define SAMPLE_RATE_HZ  16000       // Sampling rate (16 kHz)
#define N_SAMPLES       2000        // Total samples (125 ms capture)
#define CHIRP_FREQ_HZ   40000       // Transmitter frequency (40 kHz)
#define CHIRP_DURATION_US 100       // Transmit pulse width (100 µs)
```

Student Notes: Configuration parameters for your sensor:

Sampling rate: _____ Hz (suggest 8–16 kHz)

N_SAMPLES: _____ (at least $5 \times 343/16000 = 0.107$ s for 1.8 m max)

Chirp frequency: _____ Hz (typically 40 kHz)

Chirp duration: _____ µs (10–100 µs, narrower is better)

Testing the Transmitter and Receiver

1. Upload `ChirpGenerate.ino` with your configuration.
2. In the Arduino Serial Monitor (115200 baud), type `t` to trigger a single chirp transmission.
3. You should see the LED on D13 flash, indicating transmission started.
4. Place an object (book, wall) about 10 cm away from the transducer.
5. Type `r` to trigger echo recording. The Arduino will:
   - Transmit the configured chirp.
   - Record N_SAMPLES from A0 at SAMPLE_RATE_HZ.
   - Stream data to Serial Monitor in the format: `sample_index,adc_count`.
6. In MATLAB, run the provided serial reader to capture this data.

Checkpoint (5/8): Show TA at least one successful echo capture from Serial Monitor (visible peak in ADC values around the predicted sample index).

Student Notes: Student notes from echo testing:

8.3 Part 3: MATLAB Data Analysis

Objective: Import the raw echo data, compute cross-correlation, extract the peak delay, and convert to distance.

Template: Use the provided `Lab06_Serial_Reader.m` skeleton. Fill in the analysis blocks.

Task 1: Plot Transmitted Chirp and Received Echo

The skeleton provides the transmitted chirp stored in `x` and the received echo in `y`. Create a figure with two subplots:

1. **Top subplot:** Transmit signal $x[n]$ versus sample index. Label: "Transmitted Chirp".
2. **Bottom subplot:** Received signal $y[n]$ versus sample index. Label: "Received Echo".

Add title with your name/ID, axis labels, and grid.

[INSERT EXAMPLE MATLAB FIGURE: Transmitted chirp (clean sinusoid or swept chirp) and received echo (same chirp delayed and attenuated)]

Student Notes: Student notes on raw data:

Task 2: Compute Cross-Correlation and Locate Peak

Use MATLAB's built-in `xcorr()` function:

```
% Compute cross-correlation
[R_xy, lags] = xcorr(x, y);

% Find the peak
[peak_value, peak_index] = max(R_xy);
peak_lag_samples = lags(peak_index);

% Convert to time delay
Ts = 1 / Fs;  % sampling period (Fs is your sampling rate)
delta_t = peak_lag_samples * Ts;  % time delay in seconds
```

Create a figure showing the cross-correlation $R_{xy}[k]$ versus lag k (in samples). Mark the peak with a red circle or annotation.

[INSERT EXAMPLE MATLAB FIGURE: Cross-correlation function with peak clearly marked, showing lag values on x-axis]

Student Notes: Cross-correlation results:

Peak sample lag: _____ samples

Peak correlation value: _____

Time delay: _____ ms

Task 3: Convert Time Delay to Distance

Using the time-of-flight formula from pre-lab:

$$d = \frac{v_{\text{sound}} \cdot \Delta t}{2} = \frac{343 \text{ m/s} \times \Delta t}{2}. \tag{10}$$

Student Notes: Distance calculation:

Measured time delay $\Delta t =$ _____ ms

Speed of sound $v = 343$ m/s

Measured distance $d = v \times \Delta t/2 =$ _____ cm

Checkpoint (6/8): Show TA your three MATLAB figures (raw signals, cross-correlation, annotated peak) and your calculated distance for Checkpoint 2 (CP-2).

8.4 Part 4: Arduino Edge Compute Implementation

Objective: Implement a basic cross-correlation directly on the Arduino to estimate the time delay without relying on MATLAB.

Note: The Arduino has limited memory and CPU. You will implement a simplified sliding dot-product correlation that processes fewer samples and lower precision than MATLAB.

Skeleton Algorithm

The provided skeleton `EchoCapture.ino` will be extended with a correlation block:

```
// After capturing echo samples into array 'received[]':
int max_correlation = 0;
int peak_lag = 0;

// Compute correlation for lags 0 to MAX_LAG
for (int lag = 0; lag < MAX_LAG; lag++) {
    int correlation = 0;

    // Dot product: sum(transmit[n] * received[n+lag])
    for (int n = 0; n < CHIRP_SAMPLES; n++) {
        if (n + lag < N_SAMPLES) {
            correlation += transmit[n] * received[n + lag];
        }
    }

    // Track the maximum
    if (correlation > max_correlation) {
        max_correlation = correlation;
        peak_lag = lag;
    }
}

// Convert peak lag to time delay
float delta_t_arduino = (float)peak_lag / SAMPLE_RATE_HZ;
float distance_arduino_cm = 343.0 * delta_t_arduino / 2.0 * 100.0;
```

Student Notes: Arduino edge compute results:

Peak lag (samples): _____

Time delay estimate: _____ ms

Distance estimate (Arduino): _____ cm

Distance estimate (MATLAB): _____ cm

Difference: _____ cm

Checkpoint (7/8): Upload the extended Arduino code with the correlation block. Verify that the computed distance is printed to Serial Monitor. Compare with MATLAB result.

8.5 Part 5: Distance Validation Against Physical Measurements

Objective: The TA will place the sensor at several known distances (measured with a physical ruler) and verify your sensor's accuracy.

1. The TA marks 5+ reference distances on the bench: e.g., 10 cm, 20 cm, 30 cm, 50 cm, 100 cm, etc.
2. For each distance, you trigger the Arduino to record an echo.
3. You run the MATLAB analysis to compute the measured distance.
4. You record the measured and actual distances in a table.
5. You compute error: Error = |Measured − Actual|/Actual × 100%.

Table 3. Distance Validation Results

Test	**Actual Distance (cm)**	**MATLAB Distance (cm)**	**Arduino Distance (cm)**	**Error (%)**
1	_____	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____
4	_____	_____	_____	_____
5	_____	_____	_____	_____

Checkpoint (8/8): TA measures physical distances with ruler and documents 5+ test points. Student runs MATLAB analysis for each test. Checkpoint 3 (CP-3) requires accuracy $< 10\%$ for at least 3 of 5 tests.

9 Post-Lab Analysis and Deliverables

9.1 Required Discussion Questions

Pre-lab Deliverable (2/2): Answer each question in 3–6 complete sentences, supported by numerical evidence from your lab data. Attach this to your submission.

1. **Edge vs. Cloud Trade-Off:** Your MATLAB analysis and Arduino analysis produced slightly different distance estimates. Explain why the Arduino result may differ (consider: sample precision, window size, correlation window truncation, numerical rounding). Which result do you trust more, and why?
2. **SNR and Detection Limits:** Plot the received echo signal $y[n]$ and estimate the noise floor (the RMS value of the signal when no echo is present). At what maximum distance does the SNR become too poor for reliable peak detection? Support with calculations or observations from your data.
3. **Sampling Rate Impact:** You chose a sampling rate (e.g., 16 kHz). How does this rate compare to the Nyquist criterion for a 40 kHz transmitter? Could a lower sampling rate (e.g., 8 kHz) still work? Why or why not?
4. **Real-World Application:** For the stratospheric platform mission described in the Real-World Context, would you recommend edge processing (Arduino) or ground processing (MATLAB via satellite link)? Consider: latency (how quickly you need the measurement), bandwidth (satellite link cost), power consumption, and accuracy. Justify your recommendation with technical reasoning.
5. **Improvements to Edge Processing:** What simple algorithmic improvements could you make to the Arduino implementation to increase its accuracy without significantly increasing code complexity? (Examples: interpolation, windowing, adaptive thresholding.)

Student Notes: Space for discussion answers (or attach separate sheet with name/ID):

9.2 Final Submission Checklist

Before submitting, verify that your submission includes **all of the following**:

- ☐ **Pre-lab calculations:** Handwritten time-of-flight derivation and reference delays for 10–100 cm, with name/ID.
- ☐ **Hardware photos:** Clear breadboard setup showing transmitter, receiver, coupling capacitor, and connections.
- ☐ **Serial Monitor evidence:** Screenshot showing successful echo data from Arduino (at least 10 lines of sample/ADC values).
- ☐ **MATLAB Figure 1:** Raw transmitted chirp and received echo, with axis labels, legend, and your name/ID in title.
- ☐ **MATLAB Figure 2:** Cross-correlation function $R_{xy}[k]$ with peak clearly marked/circled and lag values annotated.
- ☐ **MATLAB Figure 3:** Distance validation plot (Actual vs. Measured for your 5+ test points), with error bars or error annotation.
- ☐ **Distance validation table:** Printed/typed table (Table 3) with actual, MATLAB-measured,

and Arduino-measured distances for all tests.

- ☐ **Discussion answers:** Written responses to the five discussion questions with numerical support and your name/ID.
- ☐ **Arduino code appendix:** Print key sections of your `EchoCapture.ino` with the correlation block visible.
- ☐ **MATLAB code appendix:** Print the key analysis sections (xcorr computation, peak detection, distance calculation).
- ☐ **TA sign-off page:** See Table 4 below, completed with TA initials at each checkpoint.

9.3 TA Checkpoint Sign-Off

Table 4. TA Checkpoint Sign-Off (Complete as Lab Progresses)

CP	Requirement	Target Time	TA Initials & Date
CP-1	Pre-lab calculations verified. Hardware wiring inspected and approved.	0:00–0:25	_____
CP-2	Arduino successfully captures echo and streams data. MATLAB plots show raw signals and cross-correlation with marked peak.	0:25–1:15	_____
CP-3	Distance validation tests completed (5+ distances). Accuracy $< 10\%$ for ≥ 3 tests. Arduino edge compute results documented.	1:15–2:00	_____

Lab Complete: Once all three checkpoints are signed off and the final submission checklist is complete, your lab is done. Congratulations on implementing time-of-flight estimation!

Appendix A: Cross-Correlation Mathematics (Reference)

A.1 Definition and Interpretation

The discrete-time cross-correlation between signals $x[n]$ and $y[n]$ is:

$$R_{xy}[k] = \sum_{n=0}^{N-1} x[n] \cdot y[n+k], \tag{11}$$

where k is the lag (time shift in samples).

Interpretation: $R_{xy}[k]$ measures the similarity between $x[n]$ and a delayed version of $y[n]$. Large positive values indicate strong similarity; zero or negative values indicate poor alignment.

A.2 Time-of-Flight Application

If $x[n]$ is the transmitted chirp and $y[n]$ is the received echo (with time delay τ in seconds), then the cross-correlation peaks at:

$$k_{\text{peak}} = \tau \cdot f_s, \tag{12}$$

where f_s is the sampling rate. Once k_{peak} is found, the time delay is:

$$\tau = \frac{k_{\text{peak}}}{f_s}, \tag{13}$$

and the distance is:

$$d = \frac{v \cdot \tau}{2}. \tag{14}$$

A.3 Why Cross-Correlation Works for Noisy Signals

Cross-correlation is a **matched filter**. If the received signal is $y[n] = A \cdot x[n - k_0] + w[n]$ (where A is attenuation and $w[n]$ is noise), then:

$$R_{xy}[k] = A \sum_n x[n]x[n + k - k_0] + \sum_n x[n]w[n + k]. \tag{15}$$

The first term peaks when $k = k_0$ (the noise term averages to near zero for uncorrelated noise). This is why cross-correlation is robust to noise.

Appendix B: Arduino PWM Configuration and ADC Sampling

B.1 Generating a 40 kHz Signal on D9/D10

The Arduino Uno has two PWM channels on pins D9 and D10 (Timer1, 16-bit). To generate 40 kHz:

```
// Set Timer1 to Fast PWM mode, prescaler 1, TOP = 400
// This gives: f_PWM = 16 MHz / 1 / 400 = 40 kHz
TCCR1A = 0b10100011;  // Fast PWM, output on D9
TCCR1B = 0b00001001;  // Prescaler = 1
ICR1 = 400;           // Period = 400 counts
OCR1A = 200;          // Duty cycle = 50%
```

The provided skeleton handles this; you only set the frequency constant.

B.2 High-Rate ADC Sampling

The Arduino's standard `analogRead()` is slow ($\sim 100\mu$s per sample). To achieve 16 kHz sampling, you need to configure the ADC manually:

```
// Set ADC prescaler to 32 (2 MHz clock)
// This gives ~130 kHz ADC sampling, fast enough for 16 kHz acquisition
ADCSRA = 0b11100101;  // Enable ADC, prescaler = 32
ADMUX = 0b01000000;   // Select A0, reference = AREF
```

The skeleton code handles this. You only adjust `SAMPLE_RATE_HZ` in the configuration.

B.3 Memory Constraints

The Arduino Uno has 2 KB of SRAM. Two large arrays quickly consume this:

- Transmit chirp: 400 samples = 800 bytes.
- Received echo: 2000 samples = 4000 bytes.
- Total: 4800 bytes, exceeds SRAM!

Solution: Store the transmit chirp in program memory (PROGMEM) and limit the echo capture to 1000 samples.

Appendix C: MATLAB Correlation and Visualization Templates

C.1 Data Import from Serial

The provided `Lab06_Serial_Reader.m` handles serial communication:

```
% After serial import:
samples = data(:, 1);  % sample index
adc_counts = data(:, 2);  % raw ADC (0-1023)

% Convert to voltage
Vref = 5.0;
voltage = (adc_counts / 1023) * Vref;

% Create time axis
Fs = 16000;  % sampling rate in Hz
Ts = 1 / Fs;
time_s = (0:length(voltage)-1) * Ts;
```

C.2 Cross-Correlation Computation and Peak Detection

```
% Compute cross-correlation
x_normalized = x / max(abs(x));  % normalize to [-1, 1]
y_normalized = y / max(abs(y));

[R_xy, lags] = xcorr(x_normalized, y_normalized);

% Find the peak
[peak_val, peak_idx] = max(R_xy);
peak_lag = lags(peak_idx);

% Convert to time delay
Ts = 1 / Fs;
delta_t = peak_lag * Ts;

% Convert to distance
v_sound = 343;  % m/s
distance_m = v_sound * delta_t / 2;
distance_cm = distance_m * 100;

fprintf('Peak lag: %d samples\n', peak_lag);
fprintf('Time delay: %.3f ms\n', delta_t * 1000);
fprintf('Distance: %.1f cm\n', distance_cm);
```

C.3 Plotting Cross-Correlation with Annotations

```
% Plot cross-correlation with peak marked
figure;
plot(lags, R_xy, 'b-', 'LineWidth', 1.5);
hold on;
plot(peak_lag, peak_val, 'r*', 'MarkerSize', 15);  % mark peak
```

```
xlabel('Lag (samples)', 'FontSize', 12);
ylabel('Correlation', 'FontSize', 12);
title(sprintf('Alice Smith (A123) - Cross-Correlation, Peak at %d samples', peak_lag), 'FontSi
grid on;
legend('xcorr', 'Peak', 'FontSize', 11);
hold off;
```

C.4 Distance Validation Plot

```
% Plot actual vs. measured distances
actual_distances = [10, 20, 30, 50, 100];  % cm
measured_distances = [10.2, 19.8, 31.5, 49.5, 98.0];  % cm

figure;
plot(actual_distances, actual_distances, 'k--', 'LineWidth', 2, 'DisplayName', 'Perfect');
hold on;
plot(actual_distances, measured_distances, 'b*', 'MarkerSize', 12, 'DisplayName', 'Measured');
xlabel('Actual Distance (cm)', 'FontSize', 12);
ylabel('Measured Distance (cm)', 'FontSize', 12);
title(sprintf('Alice Smith (A123) - Distance Validation'), 'FontSize', 12);
grid on;
legend('FontSize', 11);
xlim([0, max(actual_distances)*1.1]);
ylim([0, max(actual_distances)*1.1]);
hold off;

% Compute RMS error
error = abs(measured_distances - actual_distances) ./ actual_distances * 100;
rms_error = sqrt(mean(error.^2));
fprintf('RMS Error: %.2f %%\n', rms_error);
```

Appendix D: Troubleshooting Guide

Hardware and Measurement Issues

1. **Transmitter produces no sound (no vibration felt).**
   - Verify D9 is outputting a 40 kHz signal using an oscilloscope or frequency counter.
   - Check the coupling capacitor polarity and continuity.
   - Verify the transmitter element is not damaged (use a multimeter to check resistance, typically 1–10 kΩ).
   - Try a different D9/D10 pin; some boards have hardware conflicts.
2. **Receiver ADC output is always 512 (mid-rail) or flat.**
   - Check that the pre-amplifier is powered (check 5V at the op-amp power pins).
   - Verify the receiver element is connected to the pre-amplifier input.
   - Manually input a small signal (e.g., from a function generator) to the pre-amp to test gain.
   - If gain is very high, you may be saturating the ADC; reduce pre-amp gain if adjustable.
3. **Echo is visible on oscilloscope but not in Arduino ADC data.**
   - The ADC coupling capacitor may be blocking the signal. Check DC bias of the pre-amp output (should be 2.5 V, mid-rail).
   - Verify the ADC sampling is actually running (check serial output for increasing sample indices).
   - The echo may be arriving outside your capture window. Increase `N_SAMPLES` or reduce `CHIRP_DURATION_US`.
4. **Measured distances are off by a constant offset (e.g., always 5 cm too high).**
   - This may be due to a systematic delay in your hardware (e.g., pre-amp propagation delay, coupling capacitor settling).
   - Perform a calibration: measure a known distance and compute a correction factor.
   - In MATLAB: `distance_corrected = distance_measured - offset`.

Arduino Acquisition Issues

1. **Serial Monitor shows no output after sending trigger.**
   - Verify the baud rate is 115200 in both Arduino IDE and MATLAB.
   - Check that the USB cable is properly connected.
   - Try uploading the sketch again (sometimes the serial port loses connection).
   - Make sure `Serial.begin(115200)` is in the setup.
2. **ADC data is very noisy (scattered values).**

- This is normal for breadboards. Add a 10 µF smoothing capacitor across the ADC pin and GND.
- Reduce the sampling rate slightly to allow the ADC more settling time.
- In MATLAB, apply a median filter: `y_smooth = medfilt1(y, 5)` before correlation.

3. **Arduino cross-correlation result is very different from MATLAB.**
   - Check that both are using the same transmit and receive signals (not different captures).
   - Verify the correlation window size: Arduino may be truncating the echo.
   - Ensure Arduino data types are not overflowing: use `long` or `float` instead of `int`.
   - Print intermediate values (e.g., correlation at each lag) to debug.

MATLAB Analysis Issues

1. **xcorr() produces a very small peak or multiple peaks.**
   - Normalize both signals before correlation to improve SNR.
   - Check that the received signal is indeed a delayed replica of the transmit signal. If shapes don't match, the system may have distortion.
   - Use `[R_xy, lags] = xcorr(x, y, 'coeff')` to get correlation coefficient (normalized by signal energy).
2. **Peak lag is 0 or negative (indicates echo arrived before transmission).**
   - This is a sign of data corruption or incorrect alignment. Verify the transmit and receive signals are from the same experiment.
   - Check that you are not accidentally flipping the signal order (should be `xcorr(transmit, received)`, not `xcorr(received, transmit)`).
3. **Distance measurement jumps wildly between successive measurements.**
   - The SNR may be too low. Check: does the cross-correlation peak stand out clearly, or is it buried in noise?
   - Add pre-filtering: `y_filtered = bandpass(y, [39000 41000], Fs)` to isolate the 40 kHz component.
   - Ensure the object is not moving between measurements.

When to Ask for TA Help

- If no echo is visible on an oscilloscope (suggests transmitter failure).
- If Arduino data is completely flat or shows ADC clipping (saturation).
- If measured distances are consistently off by $> 20\%$, even after calibration attempts.
- If you suspect hardware damage (components are hot, board smells burned).

References and Inspiration

This laboratory explores cross-correlation as a fundamental signal processing technique for time-delay estimation, grounded in both classical matched filtering theory and contemporary embedded signal processing.

University Course Inspirations

Rice University ELEC 241: Correlation and Matched Filtering on Embedded Systems

Lab 06 is inspired by Rice's hands-on approach to cross-correlation on microcontrollers. Rice emphasizes that understanding correlation requires implementing it yourself—hand-computing small correlations, then comparing a low-precision Arduino implementation to MATLAB's high-precision `xcorr()` function. This lab follows Rice's philosophy of making correlation visceral through edge computing constraints and hardware-software trade-offs.

MIT 6.003 / 6.341: Detection and Estimation Theory

MIT's treatment of matched filtering and cross-correlation as the optimal signal detector under noise provides the theoretical foundation. The connection between correlation and maximum-likelihood detection informs the signal recovery methodology.

Georgia Tech ECE 2026: Real-Time Signal Detection

Georgia Tech's approach to detecting signals in noise via correlation-based methods and the practical measurement of SNR informs this lab's emphasis on quantifying accuracy under realistic conditions.

Duke University ECE 280: Lab Modernization

This lab is part of Duke's redesign emphasizing that correlation is not an abstract mathematical operation but the foundation of radar, sonar, communications, and scientific instrumentation.

References

- A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010. Chapter on correlation and Wiener filtering.
- S. Haykin, *Adaptive Filter Theory*, 4th ed. Prentice Hall, 2002. Chapter 2: Wiener Filters (theory of optimal correlation-based detection).
- S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, 2nd ed. California Technical Publishing, 1997. Chapters on correlation and matched filtering.
- J. H. McClellan, R. W. Schafer, and M. A. Yoder, *DSP First: A Multimedia Approach*, 2nd ed. Pearson, 2015.